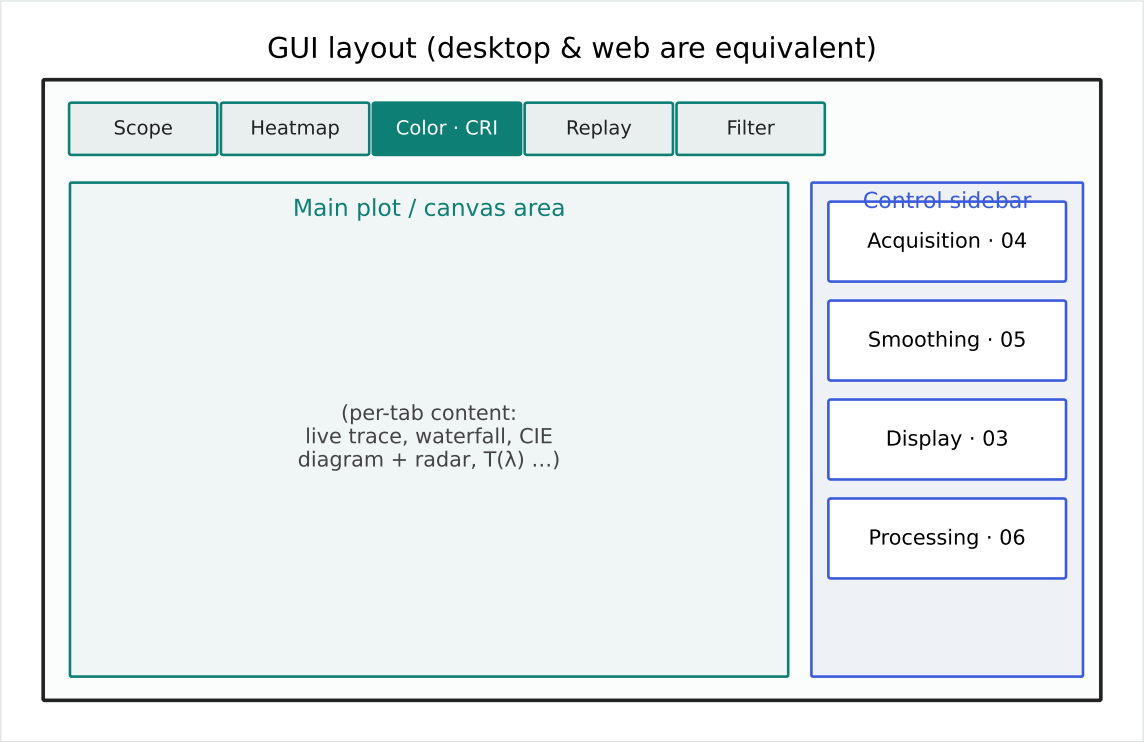
Technical & User Reference Manual

Multi-Device Spectrum Analyzer

PASCO PS-2600A • Ocean HDX • ASEQ LR-2T • Demo



A PyQt6 desktop application and FastAPI / WebSocket web client for live spectroscopy, colorimetry (CIE / CRI / TM-30 / CQS) and optical-filter characterization across four spectrometer back-ends that share a single, device-agnostic science core.

Generated June 10, 2026 • English

Contents

1 Overview

This program turns four different spectrometers into one consistent instrument. The same window (and the same numbers) work whether you have a PASCO PS-2600A CCD on USB, an Ocean Optics HDX, an ASEQ/Lasertack LR-2T, or no hardware at all (the built-in *Demo* (*virtual*) device). Every USB protocol was reverse-engineered from scratch; there are no vendor SDKs in the loop.

There are two front-ends that share one science core:

- **Desktop** — `PS-2600A_Pro.py`, a PyQt6 + pyqtgraph GUI.
- **Web** — `web_server.py`, a FastAPI + WebSocket server with a browser client (`index.html` / `app.js` / `styles.css`, a uPlot scope and canvas-drawn heatmap/CIE/radar).

Because both call the identical engines (`color_science.py`, `filter_analysis.py`), a report exported from the web and one exported from the desktop contain the same values.

What it does: live scope with overlays (CSV background reference, freeze-to-grey, difference-vs-freeze, peak-hold envelope, persistence/afterglow); a time-lapse heatmap (waterfall); CIE 1931 colorimetry (CCT, Duv, x, y, u', v', CRI Ra + R1–R15); IES TM-30-18 (Rf, Rg, 16 hue bins, colour-vector graphic, 99-sample fidelity) and CQS (Qa/Qf/Qg); optical-filter characterization (transmission, type, peak/centre λ, FWHM, 50 % edges, 10–90 % edge steepness, blocking, optical density); PDF/PNG/CSV report export; a 25-spectrum reference library; dark-current correction; in-app calibration (wavelength + response-profile wizards, per-device selection); auto-exposure; a simple fast-preview mode; optional temporal and spatial smoothing; and (web) a CSV waterfall replay player.

1.1 How to run

```
python PS-2600A_Pro.py                      # desktop GUI (entry point)
python web_server.py        # headless web server (FastAPI + WebSocket)
set FP_DEBUG=1 && python web_server.py# Windows: per-frame fast-preview
log
```

Dependencies (`requirements.txt`): PyQt6, pyqtgraph, numpy, scipy. Optional but recommended:

- **matplotlib** — CIE diagram fill and the PDF/PNG report renderers. Without it, CSV export still works; PDF/PNG return a hint.

- **colour-science** (`pip install colour-science`) — enables the IES TM-30-18 and CQS sub-tab and `/api/tm30`. Everything else (CRI, CIE, reports minus TM-30) is unaffected if it is missing.
- **pyusb + libusb-package** — Ocean HDX. **hid** — LR-2T. PASCO needs WinUSB (via Zadig) on Windows.

The Demo device needs only numpy, so the application is always usable for learning and testing with no hardware attached.

2 System architecture

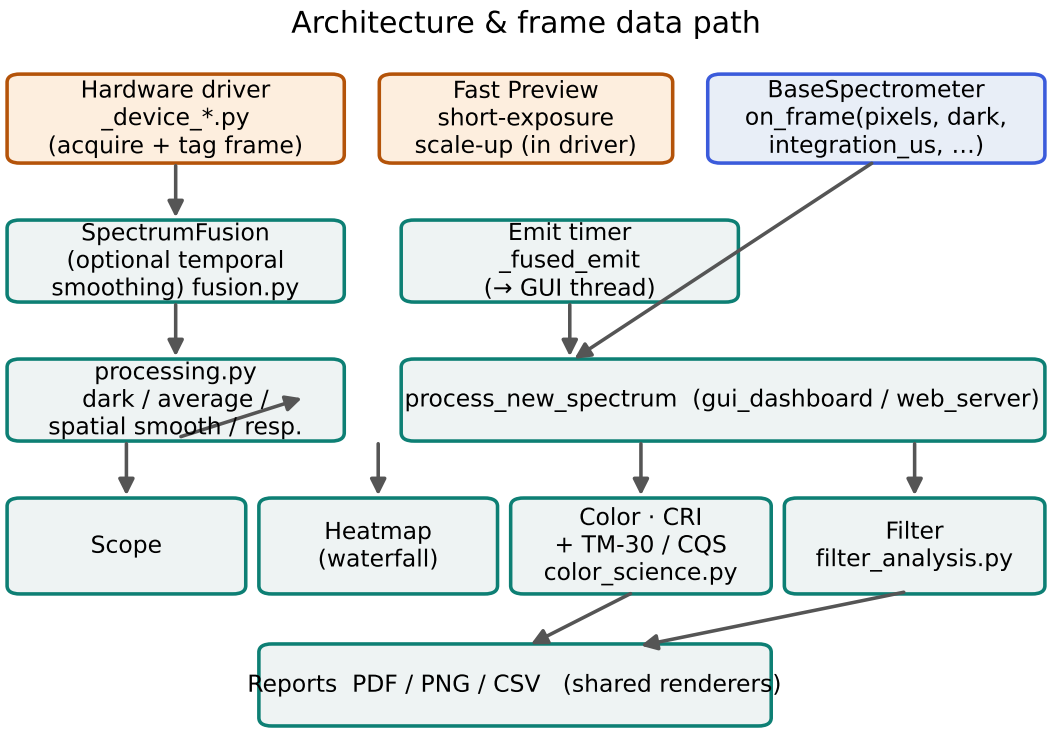


Figure 1: Module layout and the frame data path: a driver acquires and tags a frame, optional temporal smoothing and an emit timer feed a single `process_new_spectrum` entry point, which fans out to the tabs and the shared report renderers.

2.1 Modules

File	Responsibility
`PS-2600A_Pro.py`	Desktop entry point.
`spectrometer_core.py`	App-wide constants, the reference library, context help. *No device code.*

File	Responsibility
`calibration_utils.py`	Generic polynomial pixel→ λ mapping, line fitting, response gain.
`calibration_profiles.py`	Calibration store: line lists, sub-pixel peak find, response-table computation, profile registry + per-device selection.
`device_constants.py`	Device-agnostic timing and auto-exposure ratios; global integration backstops.
`device_manager.py`	`BaseSpectrometer` abstract base class and the backend registry.
`_device_pasco.py`	PASCO: WinUSB ctypes layer + acquisition `QThread`.
`_device_lr2t.py`	ASEQ/Lasertack LR-2T: HID protocol.
`_device_ocean.py`	Ocean HDX: Ocean Binary Protocol over libusb.
`_device_demo.py`	Hardware-free virtual device (numpy only).
`_usb_linux.py`	pyusb back-end for PASCO on macOS/Linux.
`fusion.py`	Device-agnostic adaptive temporal smoothing + emit stage.
`processing.py`	Signal-processing pipeline (dark, average, spatial smoother, response).
`color_science.py`	CIE XYZ, CCT, Duv, CRI, colour report/CSV, TM-30/CQS.
`filter_analysis.py`	Optical-filter engine + filter report renderer.
`gui_dashboard.py`	Main window: scope, heatmap, overlay bar, controls, frame routing.
`gui_cie_tab.py`	Colour tab (Overview / CIE 1931 / Color Rendering / TM-30·CQS).
`gui_filter_tab.py`	Optical-filter tab.
`gui_calibration.py`	Calibration dialog (desktop): wavelength + response wizards.
`gui_theme.py`	Palettes, stylesheets, custom widgets.
`app_config.py`	JSON config singleton (`Config.get/set`) and `DEFAULT_CONFIG`.
`web_server.py`	FastAPI + WebSocket server + report/TM-30 endpoints.
`index.html / app.js / styles.css`	Web front-end.

File	Responsibility
`reference_spectra.csv`	The reference-spectrum library (semicolon-delimited).

2.2 Hard rules (why the code is shaped this way)

- **All device-specific code lives in its `_device_*.py` file.** `spectrometer_core.py` holds only app-wide constants, the reference library and context help.
- **Science/signal processing is device-agnostic** (`fusion.py`, `processing.py`, `color_science.py`, `filter_analysis.py`) — never inside a driver. Drivers acquire and tag frames; processing and science decide display and metrics.
- A new device is a new `_device_xxx.py` implementing `BaseSpectrometer`, registered in `device_manager.build_registry()`, with its config keys added to `DEFAULT_CONFIG`.
- The three hardware drivers are deliberately parallel in structure and method order, so a new one can be cloned from any of them.

2.3 The frame data path

1. The driver acquires a frame and (if Fast Preview is on) has already scaled the short exposure back up to the main-exposure level.
2. It calls `on_frame(pixels, dark, integration_us, kind, ratio)` — a single, finished, displayable frame.
3. `SpectrumFusion.submit()` optionally applies adaptive temporal smoothing (if the "Smoothing" section is enabled).
4. A free-running *emit timer* marshals the latest frame onto the GUI thread and calls `process_new_spectrum`.
5. That single entry point fans out to Scope, Heatmap, Color (+CRI/TM-30) and Filter.

The emit timer keeps the canvas fluid even when new measurements arrive slowly, but it cannot invent new data — the *new-data* rate is set by the exposure (§5.6).

3 Supported hardware & drivers

3.1 Quick reference

	PASCO PS-2600A	ASEQ LR-2T	Ocean HDX
VID / PID	0x0945 / 0x0002	0xE220 / 0x0100	0x2457 / 0x2003
Driver	WinUSB (Zadig)	HID native	libusb (libusb-package)
Pixels	3648	3653 (+29 dummy)	2068
ADC bits	12	16	16
Range	~300–1085 nm	297–981 nm	346–929 nm
Optical black	30 px	none	none
Dark method	OB correction	median 50 darkest	median 50 darkest
FP timing	trigger @ t_{short}	trigger @ t_{short}	trig mode 0 + metadata settle
Python access	ctypes / pyusb	hidapi	pyusb + libusb-package

The *Demo* (*virtual*) device (§7) is always present in addition to any of the above.

3.2 The `BaseSpectrometer` interface

Every driver subclasses `BaseSpectrometer` and exposes the same shape, so the front-ends never special-case a device:

- **Class attributes:** `PIXEL_COUNT`, `WL_MIN_NM`, `WL_MAX_NM`, `SUPPORTS_OB`, `SUPPORTS_FAST_PREVIEW`, `RESPONSE_TABLE`.
- **Instance attributes:** `min_integration_us` / `max_integration_us` carry the connected device's exposure bounds, set in `connect()` from config or hardware constants; the GUI spinbox and web clamps read them, so the exposure control always follows the connected device. `serial` holds the device serial when known.
- **Methods (in order):** `scan()` → `connect(device_id)` → `start()` → `stop()` → `disconnect()` → `set_integration_time_us(us)` → `wavelength_array` property → `calibrate_from_lines()` → hardware helpers → `_run()`.

Frame callback contract. `on_frame(pixels, dark, integration_us, kind="standard", ratio=1.0)`. Every driver forwards a *finished*, displayable frame as a standard frame with

`ratio=1.0`. With Fast Preview on, the driver has already shortened the exposure and scaled the net signal up to the main-exposure level, clipped to saturation and re-added dark. The `kind`/`ratio` parameters survive only for signature compatibility.

PASCO special pattern. `PascoPS2600A` wraps a `SpectrometerAcquisition QThread`; its runtime flags (pause, auto-exposure, fast-preview) are Python properties whose setters write into the running thread object. LR-2T and HDX run `_run()` on `self`, so plain attributes suffice.

3.3 Integration-time bounds (per device)

`device_constants.GLOBAL_MIN/MAX_INTEGRATION_TIME_US` are only an absolute sanity backstop (minimum $100\,\mu$s). The real per-device limits come from each back-end: Ocean and LR-2T read `ocean_*` / `lr2t_min/max_integration_us` from config (so the HDX can be pushed below its 6 ms default — verify per firmware that the metadata exposure follows and frames do not NACK); PASCO uses its fixed hardware constants (1 ms / 2.5 s). Each back-end publishes the resolved values through `min_integration_us` / `max_integration_us`.

4 Pixel ↔ wavelength mapping & calibration

4.1 Polynomial mapping

Pixel index i (0-based) maps to wavelength by a cubic polynomial

$$\lambda(i) = C_0 + C_1 i + C_2 i^2 + C_3 i^3,$$

implemented in `calibration_utils.poly_wavelength_array` (via `np.polyval` on the reversed coefficients). For the bench PASCO unit:

$$\lambda(i) = 75.452 + 0.3458\,i - 2.337 \times 10^{-5} i^2 + 1.226 \times 10^{-9} i^3,$$

fitted from Cadmium lines at 467.8 / 480.0 / 508.6 / 643.8 nm (Fig. 2).

4.2 Calibrating from known lines

`calibration_utils.fit_wavelength_polynomial(pixel_positions, wavelengths_nm, degree)` fits coefficients (and returns the residuals) from a set of peak-pixel / known-wavelength pairs — e.g. Neon lines at 585.2 / 640.2 / 667.8 / 703.2 nm. Use Cd, Ne or Ar discharge lamps, read the peak pixel of each known line, and fit a degree-3 polynomial.

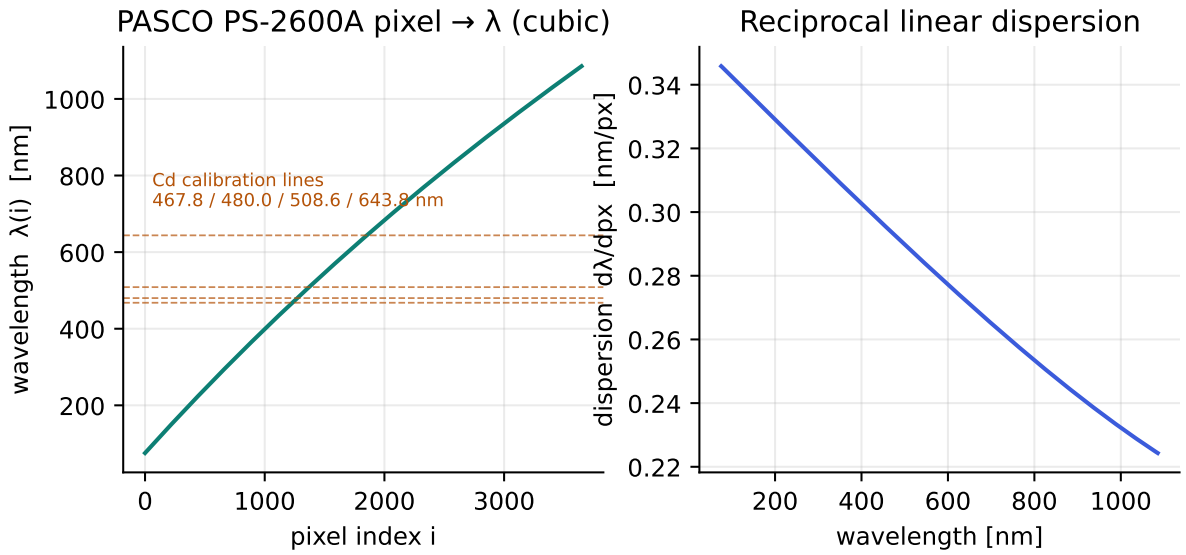


Figure 2: Left: the PASCO pixel→ λ cubic with the four Cd calibration lines marked. Right: the reciprocal linear dispersion $\mathrm{d}\lambda/\mathrm{d}px$ across the array.

A fine offset can be applied afterwards with `apply_wl_offset` (config keys `lr2t_wl_offset_nm`, `ocean_wl_offset_nm`); a positive offset shifts the displayed spectrum toward red.

4.3 The wavelength wizard (desktop)

Processing · 06 → Calibration… → Wavelength drives the fit above interactively (`gui_calibration.py`, device-agnostic helpers in `calibration_profiles.py`). Pick a lamp — curated NIST line lists for Hg, Cd, Ne, Ar, He, Kr, Xe, Na and H are built in — and *Capture & detect*: the wizard averages `calibration_capture_frames` frames (see §4.7), finds peaks to sub-pixel precision (parabolic interpolation, ∼0.03 px) and auto-pairs them with the known lines on the current axis. Edit the pixel ↔ λ table if needed, then *Fit & apply* (live on the scope) and *Save to device*, which persists the coefficients to `pasco_wl_poly_coeffs` / `lr2t_wl_poly_coeffs` / `ocean_wl_poly_coeffs` (`null` = factory axis); the driver reloads them on the next connect.

Residuals are not the whole truth. The wizard reports the max and RMS residual *at the fitted lines*, but a degree-3 polynomial constrained only in the middle of the array (e.g. by the four visible Cd lines, 467.8–643.8 nm) extrapolates badly toward the ends while still showing near-zero residual at the lines. The wizard prints an explicit *coverage warning* when the line span leaves a large part of the device range unconstrained — combine lamps (Hg + Ne + Ar) for a trustworthy full-range solution.

4.4 Per-device wavelength sources

- **PASCO** — fixed polynomial above (no on-device coefficients).

- **Ocean HDX** — wavelength and nonlinearity coefficients are read *from the device* at connect time. `ocean_use_device_wavelength` chooses device coefficients vs. a linear fallback (`ocean_wl_fallback_min/max_nm`); `ocean_nonlinearity_correction` applies the device polynomial.
- **LR-2T** — device wavelength mapping plus `lr2t_flip_pixels` (set true only if the spectrum appears mirrored after connecting) and the offset key above.

4.5 Dark-current correction

Two models, selected by `dark_mode`:

- **Optical black** (PASCO) — the 30 masked "optical black" pixels give a live dark reference; an OB slope/offset (1.00818 / -0.6377 for the bench unit) maps OB mean to the dark level.
- **Parametric** — dark $=$ bias $+$ rate $\cdot\, t$, with `dark_bias_adc` and `dark_rate_adc_per_s` (bench PASCO: 61.57 ADC and 40.94 ADC/s at 22°C).
- **HDX / LR-2T** — the dark level is estimated as the median of the `ocean_dark_estimate_pixels` (default 50) darkest pixels each frame.

Dark correction is applied in `processing.py`, not in the drivers, and is only subtracted when `dark_correction` is enabled.

4.6 Spectral-response correction & profiles

A response table (`[wavelength_nm, sensitivity]` pairs, sensitivity normalised to 1.0 at peak) becomes a per-pixel gain via `calibration_utils.build_response_gain` (gain $=$ min$(1/\text{sensitivity}, \text{max_gain})$, interpolated onto the pixel axis); multiplying raw counts by it flattens the instrument response. Unlike wavelength, this is genuinely *per fibre / optical setup* (it folds in detector QE, grating efficiency, optics *and* fibre transmission), so it is stored as *named, selectable profiles* per device.

The response wizard (desktop). *Calibration…* $\rightarrow$ *Response* measures a SMOOTH BROADBAND lamp (halogen, white LED — never a line lamp) on the device under test and divides it by a reference of the *same* lamp, captured live (e.g. on a reference spectrometer) or loaded from a CSV:

$$S(\lambda) = \frac{\text{measured}_{\text{norm}}(\lambda)}{\text{reference}_{\text{norm}}(\lambda)}.$$

Compute previews S; *Save profile* stores it and selects it. `calibration_profiles.compute_response_table` builds a *full-axis* table: measured S where the reference is reliable (above `ref_floor_frac`, default 5 % of its peak), and sensitivity $= 1.0$ (*no correction*) everywhere else, so dark gaps and bare edges are never amplified. S is peak-normalised by its 98th percentile (noise-spike-proof) and lightly smoothed (edge-coverage-normalised). Coverage < 60 % raises a *banded-source warning* — a line/band lamp (e.g. a fluorescent) is the wrong reference and yields no meaningful correction away from its bands.

Selecting a correction. The sidebar *Response correction* drop-down (it replaced the old on/off toggle) chooses per device: *None* (gain $\equiv 1$, unchanged behaviour), *Device default* (the driver's built-in `RESPONSE_TABLE`, offered only where one exists — currently the PASCO datasheet approximation), or a saved profile. Switching applies live. The selection persists per device in `response_correction_active` (`{device: selection}`); the legacy boolean `spectral_response_compensation` is kept in sync automatically, so `processing.py` needs no awareness of profiles. Profile tables live in a separate registry, `calibration_profiles_file` (`calibration_profiles.json`). The web client mirrors the dropdown (`/api/calibration/profiles`, `/select`, `/profile/delete`); profiles are *created* only in the desktop wizard and shared via the registry.

Honest limits. Outside a table's measured range `build_response_gain` returns gain 1.0 (no correction), never `max_gain` — a previous `min_floor` fill there multiplied the bare noise floor by $\sim 10\times$ across every uncovered pixel (lifting the whole red region and spiking the UV edge for a banded source). A cross-device profile matches the device to the *reference's* shape (instrument difference + fibre folded together), not to an absolute scale, and not to "pure fibre transmission" — for that, measure the same device with and without the fibre so its response cancels. Never chain calibrations (calibrating against an already-corrected reference): errors add in quadrature and clamp artefacts bake in. A self-calibration sanity check (reference = measurement) yields gain ≈ 1.0 everywhere.

4.7 Capture averaging

Every wizard capture averages `calibration_capture_frames` (default 16) *genuinely new* frames, counting noise down by $\sqrt{N}$. The capturer detects duplicate polls (the live snapshot is sampled faster than the frame rate) via array comparison and skips them, so the $\sqrt{N}$ holds regardless of frame rate. This matters more for the response correction than for the filter baseline, because a response divides two measured spectra (errors add in quadrature, amplified by $1/S$ at weak edges). Captures use the dark-corrected, response-*un*corrected spectrum — a calibration must measure the instrument, not a previous correction.

4.8 Absolute radiometric calibration

`radiometric_calibration_factor` is the scale ($\mathrm{W/m^2/nm}$ per ADC count) used to turn counts into absolute irradiance and thus lux / foot-candle readings. 0.0 means *uncalibrated* — lux displays show "–". Derive it by measuring a lamp of known spectral irradiance at a known distance and fitting the scale factor.

5 Signal-processing pipeline

All of the following live in `processing.py` / `fusion.py` and run *after* the driver, so they apply uniformly to every device.

5.1 Frame averaging

`frames_to_average` (N) averages the last N frames before display. Averaging reduces random noise by $\sqrt{N}$ without touching spectral resolution (the spectrum is static); it is the preferred noise tool.

5.2 Hot-pixel / despeckle

`hot_pixel_filter` runs a small median despeckle to remove isolated bright pixels; `smoothing_width` sets its window.

5.3 Fast Preview (simple model)

Fast Preview shortens the hardware exposure to `fast_preview_short_pct` of the main integration time, reads frames back-to-back, and scales the net signal back up:

$$\text{ratio} = \frac{t_{\text{long}}}{t_{\text{short}}}, \qquad \text{out} = \text{clip}\big((\text{pixels} - \text{dark}) \cdot \text{ratio},\, 0,\, \text{sat} - \text{dark}\big) + \text{dark}.$$

There is no long/short interleave. The scaling lives in the drivers (it needs the device saturation and dark model) — the one deliberate exception to "no processing in drivers". Per device: PASCO/LR-2T trigger at t_{short}; the HDX uses software-trigger mode 0, sets the short exposure once, settles past the stale first metadata frame, then streams and scales.

5.4 Adaptive temporal smoothing (`fusion.py`)

An optional per-pixel temporal filter (the "Smoothing" section). It smooths *stable* pixels heavily and backs off instantly where the signal changes, so peaks keep their shape and a brand-new peak appears in a single frame; it never mixes across wavelength. Parameters: `fusion_smoothing` (0–1 strength on stable pixels), `fusion_change_sigma` (deviation at which it backs off), `fusion_noise_floor` / `fusion_noise_gain` (the read- and shot-noise

model), and the emit-rate controls `fusion_emit_hz/min/max`. Off = pass-through.

5.5 Spatial (pixel-window) smoothing

`spatial_smoothing` applies a moving average of `spatial_smoothing_width` pixels along the wavelength axis (`processing.apply_spatial_smoothing`). Unlike the temporal filter it removes fixed-pattern structure (PRNU/etaloning), at the cost of broader, lower peaks — keep the window below the narrowest peak's FWHM. Off by default.

5.6 Performance reality

The *new-data* rate is $\approx 1/t_{\text{short}}$. On the HDX a fixed $\sim$25 ms host per-frame overhead (USB read + processing) sets a floor: below $t_{\text{short}} \approx 25$ ms the rate is host-limited, not integration-limited. Set `FP_DEBUG=1` to log per-frame Δt, new-vs-repeat (payload hash) and measured Hz against the $1/t_{\text{short}}$ ceiling. The emit timer can refresh the canvas faster but cannot create new measurements.

6 The reference-spectrum library

`reference_spectra.csv` holds 25 illuminant and lamp spectra used as scope background overlays, as Demo-device sources, and for teaching. It is generated automatically by `spectrometer_core.py` if absent (regenerated when the "Solar AM1.5G" column is missing — a version sentinel).

6.1 File format

A semicolon-delimited table; the first column is `Wavelength_nm`, every further column is one named spectrum:

```
Wavelength_nm;Hydrogen (H);Mercury (Hg);Sodium (Na);Neon (Ne);Argon (Ar);
Helium (He);…;Fluorescent 2-band;Fluorescent 3-band;…;LED White Cool;Solar AM1.5G
```

The bench file spans 300–1100 nm in 0.5 nm steps (1601 rows). Columns include discharge lamps (H, Hg, Na, Ne, Ar, He, Kr, Xe, Cd, Ne–Ar mix), fluorescent types (2/3/4-band), a full LED set (violet→red plus warm/neutral/cool white) and Solar AM1.5G.

6.2 How spectra are read

Two loaders, both semicolon-aware:

- `spectrometer_core.load_reference_library(target_wavelengths)` — parses the CSV into `name` → `spectrum`. If a target wavelength axis is given (the connected device's), each column is resampled onto it with `np.interp` (zero outside the source range)

and normalised to $0 \ldots 1$; otherwise the native wavelength column is returned. This drives the "Reference library" overlay selector.

- `_device_demo._load_csv_spectrum(name, wl)` — the Demo device's loader: it matches a column by name (partial match allowed), reads the `(wavelength, value)` pairs and resamples onto the demo's pixel axis, again zero outside the source range, then peak-normalises (max $= 1$).

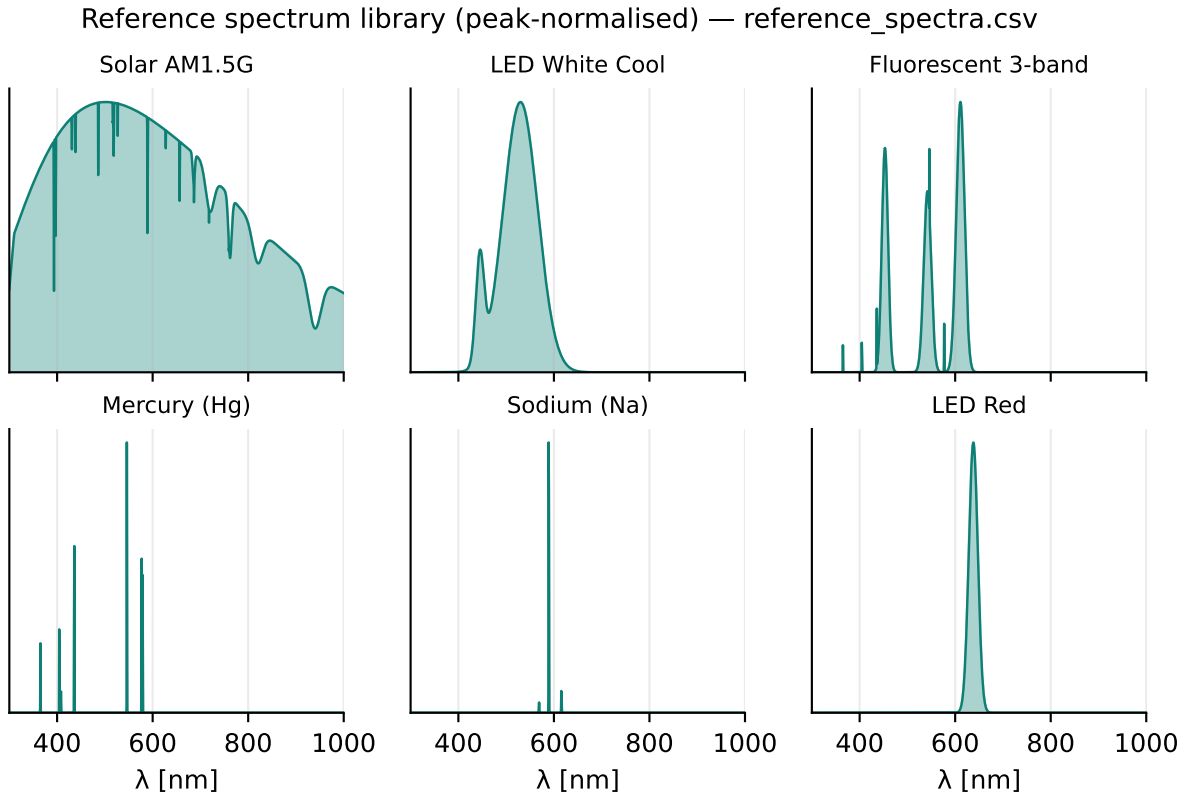


Figure 3: Six examples from the library (peak-normalised). A 2-band fluorescent, for instance, has energy only in the blue and red and is therefore a genuine off-locus magenta — see §8.3.

7 The Demo (virtual) device

`_device_demo.py` is a hardware-free back-end registered as "Demo (virtual)". It needs only numpy, is always present, and behaves like a real spectrometer (supports Fast Preview and auto-exposure) so the whole UI can be exercised with no hardware. It produces a 2048-pixel, 16-bit spectrum over 340–850 nm.

7.1 Signal model

- **Shape** — chosen by `demo_spectrum`: a built-in (`led_white`, `led_cool`, `tungsten`, `flat`) or any column name from `reference_spectra.csv` (e.g. "Solar AM1.5G", "Fluorescent 3-band", "Mercury (Hg)"), peak-normalised to 1.0.

- **Brightness vs. exposure** — the net signal scales linearly with exposure, reaching ≈ full scale at `demo_fullscale_ms` (default 1000 ms), with a *soft saturation* knee at `demo_soft_knee` (default 85 % of full scale) so the top of the range rolls over smoothly instead of hard-clipping (Fig. 4).
- **Dark** — exposure-dependent, dark $= b + c \cdot t_{\mathrm{ms}}$, where b is `demo_dark_bias` and c is `demo_dark_current` (default $500 + 1.0\,t$), with its own shot noise.
- **Noise** — read noise `demo_read_noise` (default 12 ADC) plus shot noise scaled by `demo_shot_noise` ($\times\sqrt{\text{signal}}$).

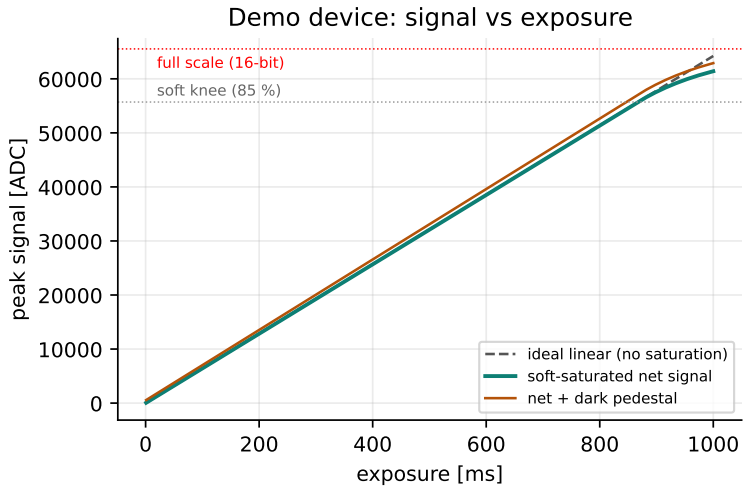


Figure 4: Demo signal vs. exposure: the ideal linear net (dashed) is bent by the soft knee at 85 % of full scale; the exposure-dependent dark pedestal sits on top.

8 The graphical interface

The desktop and web layouts are equivalent (Fig. 5): a row of tabs over a large plot/canvas area, with a collapsible control sidebar on the right.

8.1 Scope tab

The live trace (intensity in ADC vs. wavelength). Overlays (toolbar above the plot):

- **Freeze** — snapshots the current trace as a grey reference (absolute ADC). Click again to clear.
- **Difference** — shows live − frozen snapshot (signed ADC, magenta); freeze a reference first.
- **Peak-hold** — a per-pixel running maximum (yellow), like an equaliser peak meter; toggle to re-arm.

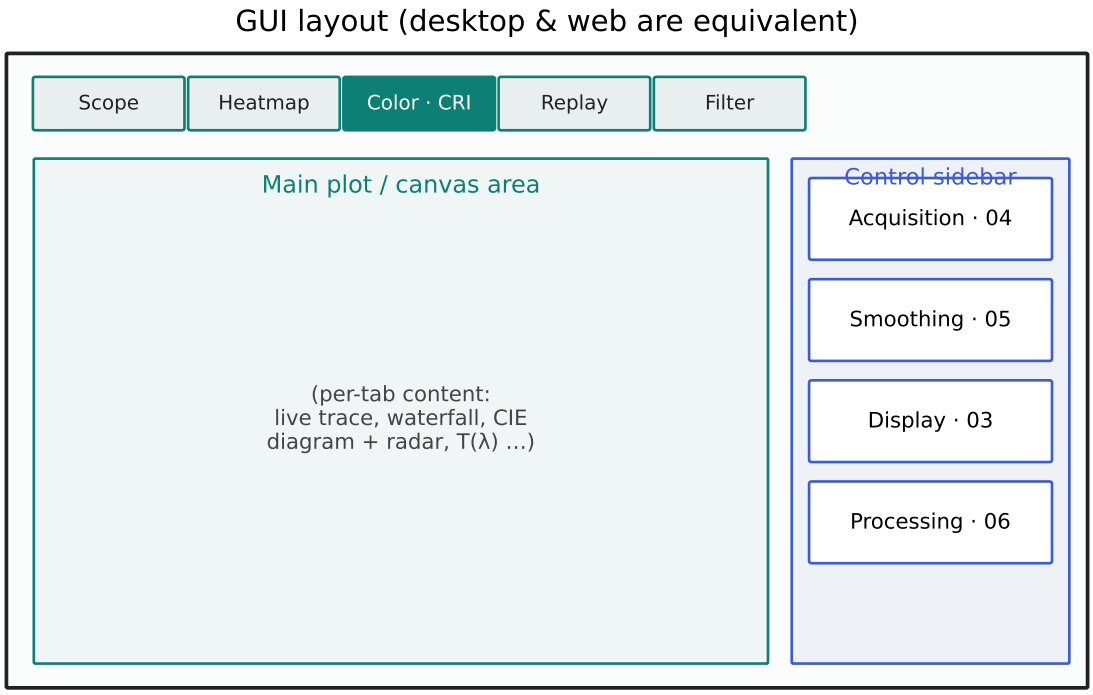


Figure 5: The window layout shared by the desktop and web front-ends: a tab bar (Scope, Heatmap, Color·CRI, Replay, Filter) over the main plot area, with the four control sections on the right.

- **Persistence** — fading echoes of the last few frames, oscilloscope-style.
- **CSV background reference** — load a 2-column (wavelength, intensity) CSV, drawn cyan and normalised to the current Y max; or pick a library spectrum via "Reference library".

8.2 Heatmap (waterfall) tab

A time-lapse image: each new spectrum becomes one coloured row. **The newest frame is at the bottom and history scrolls upward** (Fig. 6); this direction is unified across the desktop, the web heatmap and the replay player. Hovering shows a crosshair with the wavelength, ADC value and frame age, and drives the top-bar readout.

8.3 Color · CRI tab

Four sub-tabs share one measurement. *Overview* shows colorimetry cards; *CIE 1931* the chromaticity diagram with a MacAdam ellipse; *Color Rendering* the CRI radar and per-sample bars; *TM-30 · CQS* the TM-30 results. Every card has a tooltip with its definition; the metrics themselves are collected in §9.

CCT validity (off-locus guard). CCT is only physically meaningful for near-white sources close to the Planckian locus. The engine reports a number only when it is finite,

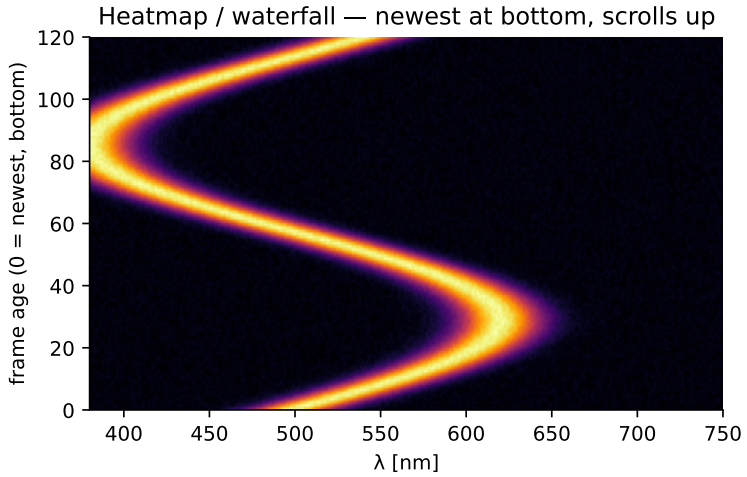


Figure 6: Waterfall convention: row 0 (newest) at the bottom, older rows above. A drifting peak therefore appears to rise.

within 1000–25000 K *and* $|\mathrm{Duv}| \leq 0.05$ (the ANSI/IES whiteness limit); otherwise CCT is shown as "–". This prevents absurd values (McCamy's approximation diverges for off-locus chromaticities). x, y, Duv and the peak/dominant wavelengths are still reported, because they correctly describe even an exotic source.

8.4 Replay tab (web)

Loads a heatmap CSV exported from the app (or a single-spectrum CSV) and replays it as a waterfall with a transport (play/scrub/speed/loop). **The current frame is drawn at the bottom with earlier frames stacked above**, matching the live heatmap.

8.5 Filter tab

Characterises an optical filter by ratioing two spectra.

1. With the *light source only* (no filter in the beam) press **Set baseline**. The app averages `filter_baseline_frames` frames (default 16) into the 100 % reference; the button shows "Averaging … k/N". Averaging the static reference cuts its noise $\sim \sqrt{N}$ without losing resolution — and that noise would otherwise enter every later $T =$ sample/reference.

2. Insert the filter. The tab plots live transmission $T(\lambda) =$ sample/reference in percent and computes the metrics.

Where the reference is below 2 % of its maximum, T is masked (the data is genuinely uncertain there). Optional display-only smoothing (`filter_display_smoothing`, default off) calms the *drawn* curve only; all metrics always run on the raw transmission.

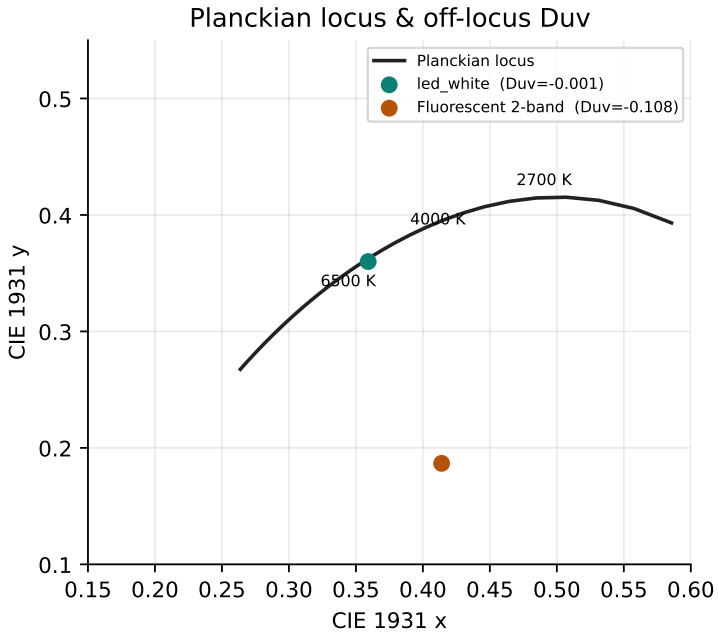


Figure 7: The Planckian locus computed by `color_science` (blackbody SPDs → XYZ → xy), with two Demo sources plotted. A near-white LED sits on the locus; the 2-band fluorescent is far below it (large |Duv|).

8.6 Control sidebar

- **Acquisition** — exposure (ms), auto-exposure, Fast Preview + short-frame %, frames to average, reference library.
- **Smoothing** — adaptive temporal smoothing strength, change-σ and display emit rate (§5).
- **Display** — X min/max (nm), auto-scale Y or a fixed Y max, snap Y to the current peak.
- **Processing** — dark correction, spatial (pixel-window) smoothing width, a *Response correction* dropdown (None / Device default / saved profile) and a *Calibration…* button opening the wavelength + response wizards (§4.3, §4.6).

9 Measurements reference

All colorimetry comes from `color_science.measure_all(wl, inten, lux_cal)`. TM-30/CQS come from `tm30_metrics` / `cqs_metrics` (via colour-science). Filter metrics come from `filter_analysis.analyze_filter`.

9.1 Colorimetry & CRI

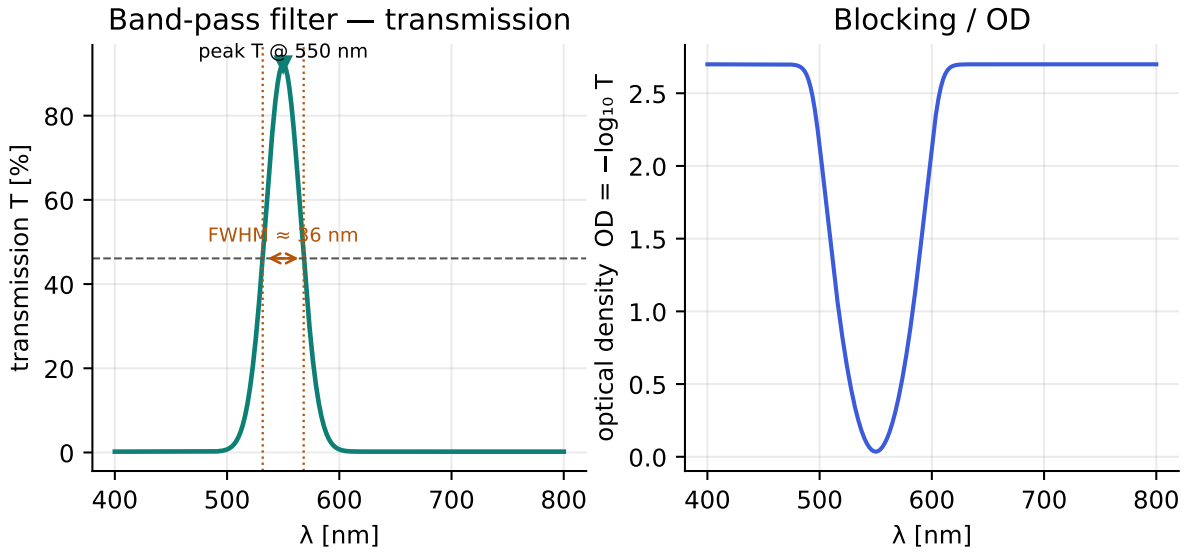


Figure 8: A band-pass example. Left: transmission with the 50 % points giving FWHM. Right: the same as optical density $\mathrm{OD} = -\log_{10} T$, which makes the blocking floor readable.

Metric	Meaning
CCT [K]	Correlated colour temperature — the blackbody whose colour most closely matches the source (McCamy approximation). "–" when off-locus (§8.3).
Duv	Signed distance from the Planckian locus on the CIE 1960 uv diagram. + = above (greenish), − = below (pinkish). $\lvert\mathrm{Duv}\rvert > 0.006$ makes CCT less meaningful; beyond ≈ 0.05 CCT is reported as "–".
x, y	CIE 1931 chromaticity coordinates.
u', v'	CIE 1976 uniform chromaticity coordinates.
CRI Ra	General colour-rendering index, the average of R1–R8. 100 = perfect; below ∼80 colours look noticeably off.
R1–R15	Per-sample colour-rendering indices (R9–R14 are saturated/skin/foliage test colours; R15 is Asian skin).
SDCM	Standard Deviation of Colour Matching — MacAdam-ellipse steps from the target white. Lower = tighter consistency.
Peak λ	Wavelength of maximum spectral intensity.
Dominant λ	Where a line from the white point through the source colour meets the spectral locus.
Centroid λ	Intensity-weighted mean wavelength (spectral centre of mass).

Metric	Meaning
FWHM	Full width at half maximum of the main peak (a bandwidth measure).
Purity	Excitation purity — 0 % = white, 100 % = monochromatic.
S/P	Scotopic/Photopic ratio — relates rod (night) to cone (day) sensitivity; higher favours low-light efficiency.
R / G / B %	Relative contribution of the red/green/blue regions to the integrated spectrum.
lux / fc	Illuminance — only when a radiometric calibration factor is set, else "–".

9.2 IES TM-30-18 & CQS

Metric	Meaning
Rf	Fidelity index (0–100) over 99 colour evaluation samples — how accurately colours are reproduced vs. a reference of the same CCT.
Rg	Gamut index — average saturation vs. the reference. ≈100 neutral, >100 more saturated, <100 duller.
Rf,hj / Rcs,hj / Rhs,hj	Per-hue-bin (16 bins) fidelity, local chroma shift and hue shift.
Colour-vector graphic	The 16-bin reference vs. test arrows on a hue circle; arrow direction/length shows hue and chroma shifts.
99-sample fidelity	Per-sample Rf,i bars, coloured by a continuous red→magenta hue ramp.
CQS Qa / Qf / Qg	Colour Quality Scale: general quality, fidelity and gamut variants.

TM-30/CQS need colour-science installed; they take ∼80 ms on a PC and are therefore throttled and computed only while their sub-tab is visible.

9.3 Optical-filter metrics

Metric	Meaning
Filter type	long-pass / short-pass / band-pass / band-stop (notch) / neutral-density, inferred from the transmission shape.
Peak T [%]	Maximum transmission, and the wavelength at which it occurs.
Centre λ / FWHM	Band centre and full width at half the peak transmission (band-pass / band-stop).
50 % edges	The cut-on / cut-off wavelengths where T crosses half-peak.
Edge steepness	10–90 % edge transition width (nm); smaller = steeper.
Avg. passband T	Mean transmission across the pass region.
Blocking min-T / OD	Deepest blocking and the optical density OD $= -\log_{10} T$ (peak and blocking-average).
Valid range	Wavelength span where the reference was strong enough to trust T.

10 Reports & export

Both apps export from the shared renderers, so numbers match exactly.

- **Colour/CRI report** (`render_color_report`) — a white-background PDF/PNG with the SPD, CRI radar (drawn in Cartesian coordinates to avoid a matplotlib polar-fill artefact), TCS-coloured R1–R15 bars, the TM-30 colour-vector graphic and 99-sample fidelity (when colour-science is present), and a full metrics table.
- **Filter report** (`render_report`) — the transmission plot and a metrics table that always lists slope/OD/FWHM/blocking ("–" when not applicable).
- **CSV** — raw metric rows for either report.
- **Parameter header** — every report carries a monospaced block with Spectrometer, Serial, Date/time, Exposure, Dark/baseline and Peak ADC, so a saved report is self-describing.

11 Web server & API

`web_server.py` (FastAPI + WebSocket) streams frames to the browser and exposes:

Endpoint	Purpose
`GET /`	Serves `static/index.html` (the web client).
`WebSocket /ws/stream`	Live frames + config push (the client mirrors `Config.all()` into `State.cfg`).
`GET /api/status`	Server / connection status.
`GET /api/backends`	Available device back-ends.
`GET /api/devices`	Scan for connected devices.
`POST /api/connect` | `/api/disconnect`	Connect / disconnect a device.
`GET /api/config` | `POST /api/config`	Read / write configuration.
`GET /api/references` | `/api/reference`	Reference-library list / fetch.
`GET /api/heatmap`	Current waterfall history.
`GET /api/export/csv`	Spectrum CSV export.
`POST /api/filter`	Live filter metrics from a (wavelengths, reference, sample) payload.
`POST /api/filter/report`	Filter report (pdf / png / csv).
`POST /api/color/report`	Colour/CRI report (pdf / png / csv).
`POST /api/tm30`	TM-30 + CQS ($\rightarrow$ {tm30, cqs, bin_colors}); 503 without colour-science.

The web client is served from a `static/` folder next to `web_server.py` (`Path(__file__).parent/"static"`); the three front-end files (`index.html`, `app.js`, `styles.css`) must live there.

12 Configuration reference

All settings live in `spectrometer_config.json`, created from `app_config.DEFAULT_CONFIG` on first run; unknown gaps are filled from defaults at load. Edit it while the app is closed, or change values from the UI (which writes back). New code keys must be added to `DEFAULT_CONFIG`.

Key	Default	Meaning
Appearance / acquisition		
`theme`	dark	"dark" or "light".
`exposure_ms`	20.0	Main integration time.
`frames_to_average`	1	Frames averaged before display.
`auto_exposure`	false	Enable auto-exposure.
`reference_library`	None	Selected overlay spectrum.
Display		
`x_min_nm / x_max_nm`	380 / 1050	Plot wavelength range.
`auto_y`	true	Auto-scale Y vs. fixed.
`y_max_adc`	4000	Fixed Y max when auto off.
`show_peaks`	false	Peak markers.
`measure_mode`	false	Measurement cursors.
Processing / dark		
`dark_correction`	false	Subtract dark.
`dark_mode`	`optical_black`	"optical_black" or "parametric".
`dark_bias_adc`	61.57	Parametric dark bias.
`dark_rate_adc_per_s`	40.94	Parametric dark rate.
`hot_pixel_filter`	false	Median despeckle.
`smoothing_width`	5	Despeckle window.
`spatial_smoothing`	false	Pixel-window moving average.

Key	Default	Meaning
`spatial_smoothing_width`	5	Window (odd, 3–51).
`spectral_response_compensation`	false	Master on/off, kept in sync by the response-correction dropdown (do not edit by hand).
`radiometric_calibration_factor`	0.0	W/m²/nm per ADC; 0 = uncalibrated.
Calibration (§4.6)		
`response_correction_active`	{}	Per-device selection `{device: None/Device default/profile}`.
`calibration_profiles_file`	`calibration_profiles.json`	Response-profile registry path.
`calibration_capture_frames`	16	New frames averaged per wizard capture.
`pasco_wl_poly_coeffs`	null	Persisted wavelength polynomial; null = factory axis.
`lr2t_wl_poly_coeffs`	null	Persisted wavelength polynomial; null = device/cal-file axis.
`ocean_wl_poly_coeffs`	null	Persisted wavelength polynomial; null = device axis.
LR-2T		
`lr2t_wl_offset_nm`	0.0	Wavelength fine-tune.
`lr2t_flip_pixels`	false	Mirror pixel order if needed.
`lr2t_min/max_integration_us`	1000 / 1e7	Exposure bounds (1 ms / 10 s).
`lr2t_discard_frames_after_itime`	1	Drop N stale frames after an exposure change.
Ocean HDX		
`ocean_pixel_count`	2068	Auto-updated from device.

Key	Default	Meaning
`ocean_use_device_wavelength`	true	Device WL coeffs vs. fallback.
`ocean_wl_offset_nm`	0.0	Wavelength fine-tune.
`ocean_wl_fallback_min/max_nm`	200 / 1100	Linear axis if WL read fails.
`ocean_nonlinearity_correction`	true	Apply device NL polynomial.
`ocean_dark_estimate_pixels`	50	Median of N darkest = dark level.
`ocean_adc_saturation`	60000	Saturation / AE emergency drop.
`ocean_ae_target_adc`	50000	Auto-exposure target peak.
`ocean_ae_deadzone`	2000	AE no-change band.
`ocean_trigger_mode`	0	0 = free-running.
`ocean_min/max_integration_us`	6000 / 1e7	Exposure bounds (6 ms / 10 s).
`ocean_spectrum_command`	metadata	`metadata` / `raw_hdx` / `buffered` / `raw_legacy`.
`ocean_usb_timeout_ms`	15000	Spectrum transfer timeout.
`ocean_loop_sleep_factor/min/max`	0.5 / .005 / .20	Standard streaming loop sleep.
`ocean_fp_loop_sleep_s`	0.001	Fast-preview yield (no t_{short} scaling).
`ocean_discard_frames_after_itime`	1	Drop N stale frames after itime change.
`ocean_fp_long_refresh_s`	3.0	FP long-reference refresh.

Key	Default	Meaning
`ocean_response_table`	null	`[[λ, sensitivity], …]` or null.
Demo (virtual)		
`demo_min/max_integration_us`	1000 / 1e6	1 ms / 1000 ms.
`demo_fullscale_ms`	1000.0	Exposure where peak ≈ full scale.
`demo_spectrum`	led_white	Built-in or a CSV column name.
`demo_dark_bias`	500.0	Fixed dark offset (counts).
`demo_dark_current`	1.0	Dark current (counts/ms).
`demo_soft_knee`	0.85	Soft-saturation knee (fraction of full scale).
`demo_read_noise`	12.0	Read-noise σ (counts).
`demo_shot_noise`	1.0	Shot-noise scale ($\times\sqrt{\text{signal}}$).
Fast Preview / smoothing		
`fast_preview_enabled`	false	Enable Fast Preview.
`fast_preview_short_pct`	10	Short frame as % of main exposure.
`fusion_enabled`	false	Adaptive temporal smoothing.
`fusion_smoothing`	0.85	Strength on stable pixels (0–1).
`fusion_change_sigma`	4.0	σ at which smoothing backs off.
`fusion_noise_floor / gain`	8.0 / 1.0	Read / shot-noise model.
`fusion_emit_hz / max / min`	0 / 60 / 2	Output rate (0 = auto).
Filter / peaks / cursors		
`filter_baseline_frames`	16	Frames averaged into the 100 % reference.

Key	Default	Meaning
`filter_display_smoothing`	0	Display-only curve smoothing window (0 = off).
`peak_savgol_window / order`	11 / 3	Savitzky–Golay peak detection.
`peak_prominence / min_height`	50 / 15	Peak thresholds (ADC).
`peak_min_distance_px / max_count`	40 / 3	Peak spacing / count.
`measure_a_nm / measure_b_nm`	500 / 600	Last cursor positions.

13 Extending: adding a new device

1. Clone the closest `_device_*.py` (they are structurally parallel).
2. Set the class attributes (`PIXEL_COUNT`, `WL_MIN/MAX_NM`, `SUPPORTS_OB`, `SUPPORTS_FAST_PREVIEW`, `RESPONSE_TABLE`) and implement the methods in the documented order.
3. In `connect()` set `min_integration_us` / `max_integration_us` (and `serial` if available).
4. Emit finished frames via `on_frame(…, kind="standard", ratio=1.0)`; if you implement Fast Preview, do the short-exposure scaling in the driver.
5. Register the class in `device_manager.build_registry()` and add any new config keys to `DEFAULT_CONFIG`.

Keep all signal/science work out of the driver — it belongs in `processing.py` / `color_science.py` / `filter_analysis.py`.

14 Troubleshooting & known non-issues

- **CCT shows "–"** — the source is off-locus ($|\mathrm{Duv}| > 0.05$); this is intended, not a fault. x, y and the wavelengths are still valid.
- **Low FPS at long exposure** — exposure-limited physics (new-data $\approx 1/t_{\mathrm{short}}$), not a bug (§5.6).

- **Response correction does nothing** — the dropdown is on *None*, or *Device default* is hidden because the driver ships no `RESPONSE_TABLE` (LR-2T / HDX). Measure a profile in the response wizard, or select one.
- **Corrected spectrum has a lifted noise floor / edge spikes** — the response reference was a line/band source (the wizard warns below 60 % coverage). Use a smooth broadband lamp; a banded source carries no information in its gaps.
- **TM-30/CQS sub-tab missing or slow** — install colour-science; on a slow host it is throttled by design. CRI/CIE/other reports are unaffected.
- **Spectrum mirrored (LR-2T)** — set `lr2t_flip_pixels` true.
- **Filter plot looks noisy** — raise `filter_baseline_frames` (average a cleaner reference) rather than smoothing $T(\lambda)$, which would distort edge/FWHM/OD metrics.

A Appendix: bench calibration constants

PASCO (serial ASQ_SPC5636146, 22°C): dark bias 61.57 ADC, rate 40.94 ADC/s; optical-black slope 1.00818, offset -0.6377. Wavelength cubic $\lambda(i) = 75.452 + 0.3458\,i - 2.337 \times 10^{-5} i^2 + 1.226 \times 10^{-9} i^3$ from Cd lines 467.8 / 480.0 / 508.6 / 643.8 nm. PASCO interface GUID `{DEE824EF-729B-4A0E-9C14-B7117D33A817}`. Bench serials: LR-2T ASQ_SPC5636146, HDX HDX00512.

This manual is generated from the source tree; when behaviour changes, regenerate it alongside HANDOVER.md / BACKLOG.md so it stays the reference of record.